

Building the Priority List

Identification & setup · build_priority_list.R

build_priority_list.R reads the two AGO reports and writes data/outputs/cleanup_priority.csv, the ranked list of who to clean up. Its one job is to **propose** targets — it never decides anything is safe to delete on its own. The three checks it prints are how you confirm it's telling the truth.

It only ever *reads* the exported CSVs; it touches nothing in AGO. Re-run it as often as you like while you tune the config.

Where it sits in the pipeline

The script builds a candidate list → IT stamps graduation status onto it and returns it → that returned sheet becomes the input to cleanup_targets.py. Storage size and the priority score **rank** the list; they don't decide eligibility.

Inputs

- The **item report** → storage, owner, share level, views, date last viewed, recycle-bin flag.
- The **member report** → Username, Email, Role, User Type, Member Categories, Last Login Date.

Both go in data/. Point the config at them:

```
ITEM_REPORT <- "data/OrganizationItems.csv"
MEMBER_REPORT <- "data/OrganizationMembers.csv"
OUTPUT_CSV <- "data/outputs/cleanup_priority.csv"
KEEP_LIST_CSV <- "data/keep_list.csv" # never-target co-ops, staff
```

It also reads data/keep_list.csv (the never-target list — co-ops, protected staff) and writes keep = TRUE for anyone on it, alongside faculty. The candidate list lands in data/outputs/; everything else the run produces goes there too.

The three checks (this is the important part)

1. Units — confirm the storage total

The storage columns carry no unit, and AGO is inconsistent. The script prints the org total it computed; it should land near **1,151 GB**. If it's off by ~1,000,000× you have bytes-vs-MB; by ~1,000×, KB-vs-MB. Fix the unit and re-run:

```
STORAGE_COL_UNIT <- "MB" # try "bytes" / "KB" / "GB" until the total is right
```

This sets the MIN_STORAGE_GB threshold's meaning, so a wrong unit silently moves the whole target line. Don't skip this check.

2. Faculty — fill in the real values

Faculty and staff are never targeted. The script prints the distinct Role, User Type, and Member Categories values in your org so you can see which ones mean “faculty/staff,” then fill them in and re-run:

```
FACULTY_ROLES      <- c("org_admin")          # from the printout
FACULTY_USER_TYPES <- c("GIS Professional Advanced")
```

The first-initial.lastname email pattern stays as a backstop, but Role/User Type is the real signal.

3. Recovery — sanity-check the total

The script reports the recoverable storage in the target set. For the aggressive cut (≥ 1 GB, 1+ year inactive) it should be near **557 GB**. Wildly off means the units or the join are wrong.

The other knobs

- MIN_STORAGE_GB / INACTIVE_DAYS — the aggressive-cut thresholds.
- OWNER_SUFFIX_REGEX — strips department suffixes off owner names before the join.
- FACULTY_CATEGORY_REGEX / FACULTY_EMAIL_REGEX — the softer faculty signals.

Output

It writes the columns cleanup_targets.py reads — final_email, final_name, status, keep, deleted, notice_date — plus diagnostics (storage, days since login, activity status, priority score). Faculty rows get keep = TRUE written in as a second backstop.

The IT hand-off (by design). This list is a *candidate* — status is set from our criteria, not confirmed graduation. Send it to IT; they adjust the status column and may reshape the sheet, and that returned sheet is what the deletion script runs on. Expect to do a little R wrangling to reconcile what comes back.